Julio Cesar Nogueira

Spring Boot 4 & Java 21

Fase 1: Teoria Profunda

Objetivo

- Explicar a "mágica" do framework
  - Por que o nosso código simplemente funciona sem instanciarmos nada manualmente?

- Por que o código funciona sem new
  - @Autowired

Ecossistema: Camadas

- JVM 21: Virtual Threads
- Maven: Gerente de Obras - construtor
- Spring Framework: Gerente de Objetos
- Spring Boot 4: Camada de Opinião

Java 21: Novidades

- Records para DTOs concisos
- Pattern Matching em switch
- Virtual Threads (Project Loom)

IoC e DI

Analogia IoC

- Tradicional: você controla new Service()
- IoC: o contêiner providencia instâncias

DI no Código

- Sem Spring: private final CalculadorService = new CalculadoraService(); (Acoplado)
- Com Spring: @Autowired CalculadoraService (Desacoplado)

Tópico 3: pom.xml

Parent

- spring-boot-starter-parent v4.0.2
- Garante compatibilidade de versões

Dependências

- spring-boot-starter-web: Tomcat, MVC, Jackson
- spring-boot-starter-test: JUnit5, Mockito, AssertJ

Properties

- <java.version>21</java.version>

Desafio: Pipeline de Limpeza

- Configurações: remove-nulls, max-rows, strategy
- Microsserviço simula limpeza de dataset

```
# Configurações customizadas para nossos Modelos de ML
app:
  model:
    default-threshold: 0.75
    version: "v4.2.0-beta"
    path: "/mnt/models/random-forest.onnx"
```

Estrutura Java

- Record CleaningConfig para mapear YAML
- CleanerService injeta config e getModelInfo()
- CleanerController com @RequestParam que usa o getModelInfo() do meu serviço

Fase 4: Atividade

O Problema

- Sensores IoT enviam 50k leituras via URL
- Existe limites para o tamanho de URL e validação

Solução (Próxima Aula)

- POST com Request Body
- DTOs e Bean Validation (@NotNull, @Min, @Email)

Material de Apoio

- psvm + Tab: main
- sout + Tab: System.out
- Alt+Insert / Cmd+N: Gerar código
- Ctrl+Alt+L: Formatar código

Perguntas?